3-2

MCP - Model Context Protocol

학습 목차

- **MCP**
  - 함수 호출의 문제점
  - MCP의 등장
  - MCP의 구성요소 (Server, Host, Client)
- **MCP Server 구현**
  - FastMCP
  - Resources, Tools, Prompts
- **MCP Client 구현**
  - LangChain으로 연결
  - get_tools

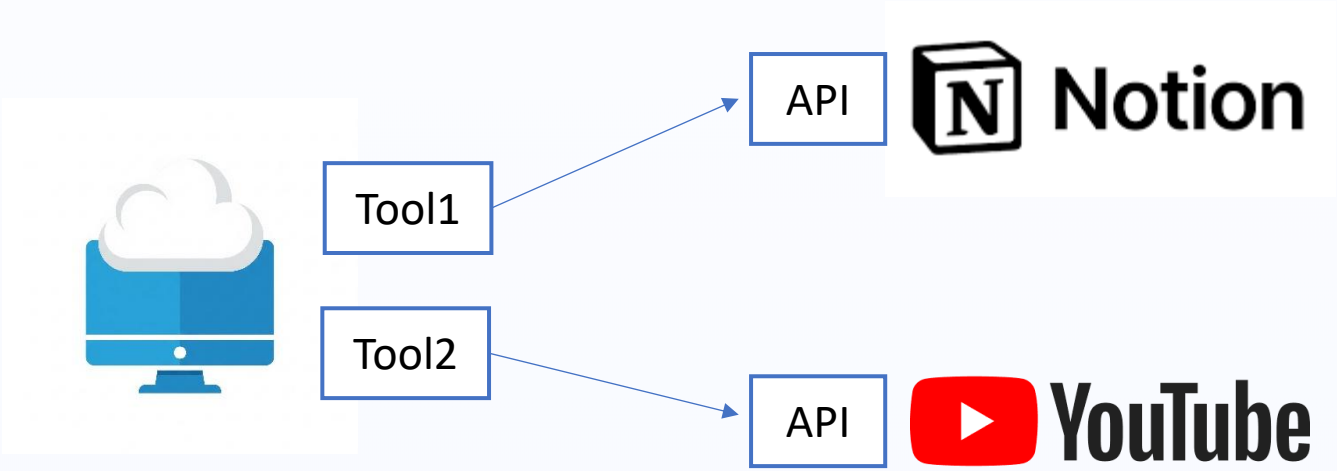
© Scopelabs. All Rights Reserved

MCP

함수 호출의 문제점

Function Calling은 LLM이 외부 함수를 사용할 수 있게 해주지만, 도구 연결 방식이 애플리케이션마다 흩어지기 쉽다.

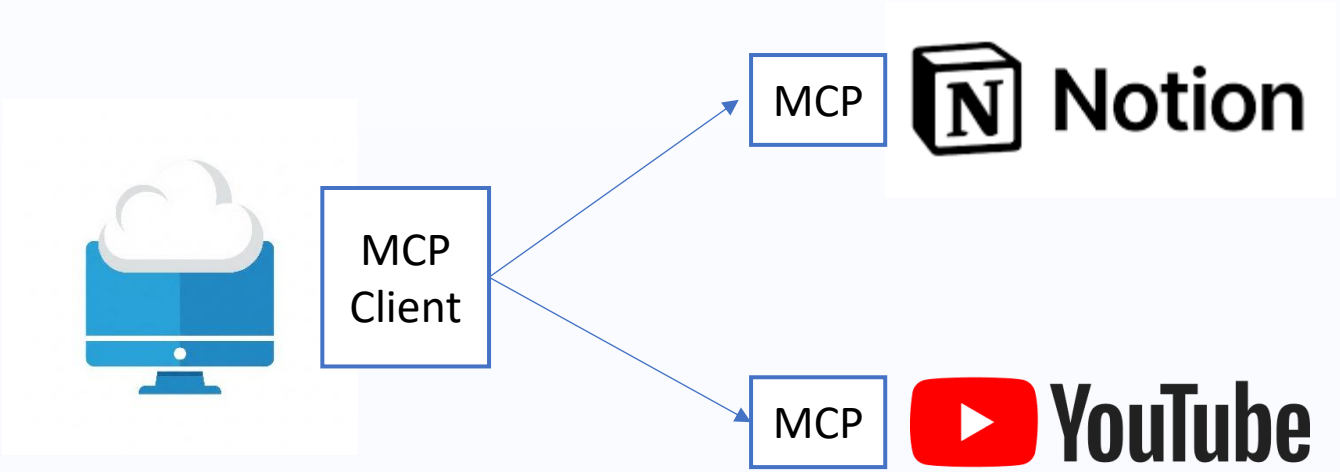
- 함수 이름, 설명, 입력 스키마를 애플리케이션 코드 안에서 직접 관리
- 도구가 늘어날수록 연결 코드와 권한 관리 코드도 함께 증가
- 같은 도구를 다른 Agent나 다른 앱에서 재사용하기 어려움
- 파일, DB, API처럼 시스템마다 연결 방식이 달라 유지보수가 복잡해짐



MCP의 등장

MCP(Model Context Protocol): LLM 애플리케이션과 외부 도구/데이터를 표준 방식으로 연결하기 위한 프로토콜

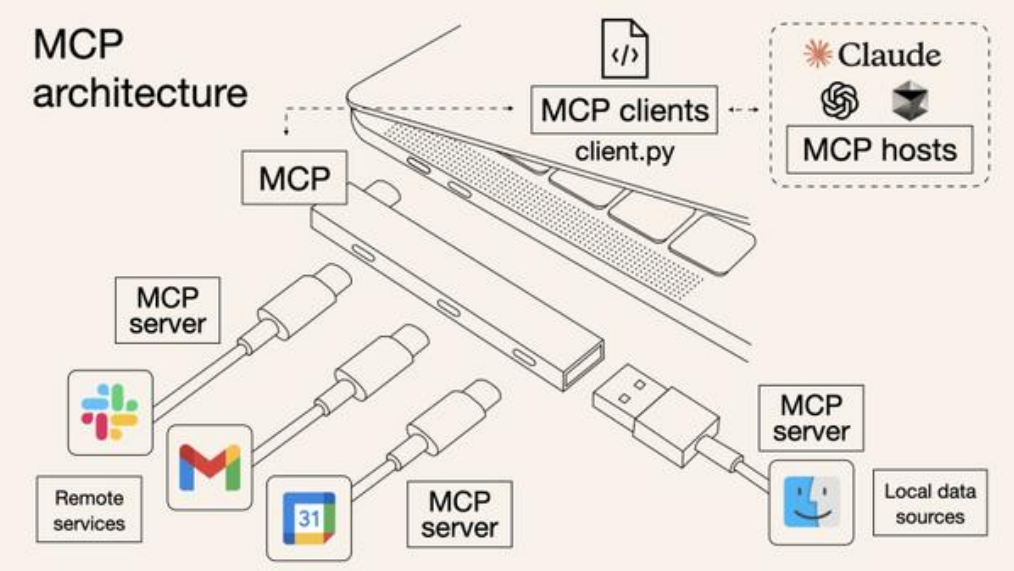
- Agent가 도구를 직접 품고 있는 대신, MCP Server가 도구와 데이터를 제공
- Host는 필요한 MCP Server에 연결해서 기능을 가져옴
- Client는 Host와 Server 사이의 통신을 담당
- 도구 구현과 Agent 실행 로직을 분리할 수 있음



MCP의 구성요소 (Server, Host, Client)

MCP는 Server, Host, Client의 관계로 이해하면 쉽다.

- Server: 외부 도구, 데이터, 프롬프트를 MCP 형태로 제공
- Host: 사용자가 실제로 사용하는 LLM 애플리케이션
- Client: Host 안에서 MCP Server와 통신하는 연결 계층



MCP Server 구현

FastMCP

FastMCP: MCP Server를 빠르게 만들기 위한 고수준 서버 구현 도구

- 데코레이터로 Tool, Resource, Prompt 정의
- HTTP/SSE 모드와 stdio 모드 실행 가능
- Claude Code 같은 MCP Host와도 stdio 방식으로 연결 가능

```
from fastmcp import FastMCP

mcp = FastMCP("AI 강의 튜토리얼 서버")

if __name__ == "__main__":
    mcp.run(
        transport="sse",
        host="0.0.0.0",
        port=8000,
    )
```

Resources, Tools, Prompts

MCP의 3대 Primitive

- Tools: LLM이 실제 행동을 수행하기 위해 호출하는 함수
- Resources: LLM이나 Host가 읽을 수 있는 데이터나 맥락 자료
  - LLM, Host 등이 등록된 서버에 대한 정보를 얻기 위함
- Prompts: 반복해서 사용할 수 있는 프롬프트 템플릿
  - 이를 활용하여 단축 명령어를 제공하기도 함

```python
@mcp.tool()
def calculate_bmi(weight_kg: float, height_cm: float) -> dict: ...

@mcp.resource("config://app-info")
def get_app_info() -> str: ...

@mcp.prompt()
def task_summary_prompt(user_name: str) -> str: ...
```

MCP Client 구현

LangChain으로 연결

LangChain에서는 `langchain_mcp_adapters`의 `MultiServerMCPClient`로 MCP Server에 연결할 수 있다.

- MCP 설정에 서버 이름, transport, url을 작성
- 하나의 Client에서 여러 MCP Server를 동시에 다룰 수 있음
- 가져온 도구는 LangChain/LangGraph의 Tool처럼 사용할 수 있음

```
MCP_CONFIG = {
    "tutorial": {
        "transport": "sse",
        "url": "http://localhost:8000/sse",
    },
}

client = MultiServerMCPClient(MCP_CONFIG)
```

`get_tools`

`get_tools`: MCP Server에 접속해서 사용 가능한 Tool 목록을 LangChain Tool 형태로 가져오는 메서드

- 서버가 제공하는 Tool 이름과 설명을 읽어옴
- LLM이 이해할 수 있는 도구 스키마로 변환
- `bind_tools`에 넘겨 Tool Calling 대상으로 사용

이후 LangChain의 다른 Tool 처럼 사용 가능!

```
client = MultiServerMCPClient(MCP_CONFIG)
tools = await client.get_tools()

llm = get_llm().bind_tools(tools)
```